

Security Findings Report

CodeAlpha Internship — Task 3: Secure Code Review

- **Application Audited:** vulnerable_app.py
- **Language:** Python 3
- **Operating System:** Windows 10
- **Tools Used:** Bandit (Static Analysis), Manual Code Review
- **Total Vulnerabilities Found:** 7
- **Date:** 2026

Summary

Severity	Count
----------	-------

● HIGH	3
--------	---

□ MEDIUM	3
----------	---

□ LOW	1
-------	---

Total	7
--------------	----------

Detailed Findings

Finding 1 — Hardcoded Credentials

- **Severity:** ● HIGH
- **Location:** vulnerable_app.py — Lines 15–18
- **Bandit Rule:** B105, B106 (hardcoded_password_string)

Vulnerable Code

```
USERNAME = "admin"
PASSWORD = "admin123"
SECRET_KEY = "mysecretkey2024"
API_KEY = "sk-abc123hardcodedkey"
```

Risk

If the source code is leaked or uploaded publicly to GitHub, attackers can immediately access sensitive credentials.

Fix Applied

```
USERNAME = os.environ.get("APP_USERNAME", "admin")
SECRET_KEY = os.environ.get("SECRET_KEY", "change-this-in-production")
API_KEY = os.environ.get("API_KEY", "")
```

Remediation

Store all credentials securely using environment variables or .env files excluded from Git using .gitignore.

Finding 2 — SQL Injection

- **Severity:**  HIGH
- **Location:** vulnerable_app.py — Lines 28–35
- **Bandit Rule:** B608 (hardcoded_sql_expressions)

Vulnerable Code

```
query = "SELECT * FROM users WHERE username = '" + username + "' AND  
password = '" + password + "'"
```

```
cursor.execute(query)
```

Risk

Attackers can bypass authentication using malicious SQL input.

Attack Example

Username: admin' OR '1'='1

Password: anything

Result: Login bypass successful.

Fix Applied

```
cursor.execute(  
    "SELECT password_hash FROM users WHERE username = ?",  
    (username,) )
```

Remediation

Always use parameterized queries or prepared statements instead of string concatenation.

Finding 3 — Command Injection

- **Severity:**  HIGH
- **Location:** vulnerable_app.py — Lines 46–48
- **Bandit Rule:** B602 (subprocess_popen_with_shell_equals_true)

Vulnerable Code

```
result = subprocess.call("ping " + host, shell=True)
```

Risk

Attackers can execute arbitrary system commands.

Windows Attack Example

Input: google.com && whoami

Result: Executes both ping and whoami commands.

Fix Applied

```
if not re.match(r'^[a-zA-Z0-9.\-]+$' , host):  
    return "Invalid host format."  
result = subprocess.call(  
    ["ping", host],
```

```
    shell=False
)
```

Remediation

Never use shell=True with user input. Validate all inputs and pass command arguments as a list.

Finding 4 — Weak Password Hashing (MD5)

- **Severity:** ☐ MEDIUM
- **Location:** vulnerable_app.py — Lines 57–59
- **Bandit Rule:** B324 (hashlib)

Vulnerable Code

```
return hashlib.md5(password.encode()).hexdigest()
```

Risk

MD5 hashes are weak and easily cracked using rainbow tables.

Fix Applied

```
salt = bcrypt.gensalt()
hashed = bcrypt.hashpw(password.encode(), salt)
return hashed
```

Remediation

Use secure hashing algorithms like bcrypt, Argon2, or scrypt.

Finding 5 — Missing Input Validation

- **Severity:** ☐ MEDIUM
- **Location:** vulnerable_app.py — Lines 68–71

Vulnerable Code

```
def calculate_discount(price, discount):
    final_price = int(price) - (int(price) * int(discount) / 100)
    return final_price
```

Risk

Invalid or malicious inputs may crash the application or manipulate logic.

Fix Applied

```
try:
    price = float(price)
    discount = float(discount)
except (ValueError, TypeError):
    raise ValueError("Invalid input.")
if price < 0:
    raise ValueError("Price cannot be negative.")

if not (0 <= discount <= 100):
```

```
raise ValueError("Discount must be between 0 and 100.")
```

Remediation

Validate all user inputs for type, range, and format before processing.

Finding 6 — Path Traversal

- **Severity:** ☐ MEDIUM
- **Location:** vulnerable_app.py — Lines 80–82

Vulnerable Code

```
def read_file(filename):  
    with open(filename, "r") as f:  
        return f.read()
```

Risk

Attackers can access sensitive Windows system files.

Windows Attack Example

Input: ..\..\Windows\System32\drivers\etc\hosts

Fix Applied

```
SAFE_DIRECTORY = os.path.abspath("./safe_files")
```

```
def read_file(filename):  
  
    safe_path = os.path.abspath(  
        os.path.join(SAFE_DIRECTORY, filename)  
    )  
  
    if not safe_path.startswith(SAFE_DIRECTORY):  
        raise PermissionError("Path traversal detected.")  
  
    with open(safe_path, "r") as f:  
        return f.read()
```

Remediation

Restrict file access to approved directories only.

Finding 7 — Information Disclosure

- **Severity:** ☐ LOW
- **Location:** vulnerable_app.py — Lines 93–99

Vulnerable Code

```
except Exception as e:  
    print("ERROR DETAILS:", e)
```

Risk

Detailed error messages may expose system paths, database details, or

internal application information.

Fix Applied

except Exception:

```
print("[LOG] Internal error occurred.")  
return "An error occurred. Please try again later."
```

Remediation

Log detailed errors internally while displaying generic messages to users.

Comparison — Vulnerable vs Secure

Vulnerability	Vulnerable Code	Secure Code
Hardcoded credentials	PASSWORD = "admin123"	os.environ.get()
SQL Injection	SQL string concatenation	Parameterized query
Command Injection	shell=True	shell=False
Weak hashing	hashlib.md5()	bcrypt.hashpw()
Missing validation	Direct int() conversion	Input validation
Path traversal	open(filename)	Path restriction
Information disclosure	Raw error messages	Generic error handling

Tools Used

- Bandit — Python static analysis tool
- Manual Code Review
- OWASP Top 10 Reference

How to Run Bandit (Windows)

```
python -m pip install bandit  
python -m bandit -r vulnerable_app.py -o bandit_report.txt -f txt  
type bandit_report.txt
```

Conclusion

The security audit of `vulnerable_app.py` identified 7 vulnerabilities across HIGH, MEDIUM, and LOW severity levels. The issues were remediated in `secure_app.py` using industry-standard secure coding practices including parameterized queries, secure password hashing, environment-based secrets management, path validation, secure subprocess handling, and improved error management.

Prepared By

Balamurugan S

CodeAlpha Cyber Security Internship — Task 3